Parallel Systems

1 Introduction

Parallel systems improve processing and I/O speeds by using a large number of computers in parallel. As datasets grow to petabyte scales, parallel databases have become essential for handling modern workloads.

- **Coarse-grain Parallelism:** Uses a small number of powerful processors (e.g., high-end servers with multicore CPUs).
- **Fine-grain/Massively Parallelism:** Uses thousands of smaller processors (nodes), typically arranged in a shared-nothing architecture in a **data center**.

2 Measures of Performance

The efficiency of a parallel system is measured by two primary metrics:

1. **Throughput:** The number of tasks completed in a given time interval.
2. **Response Time:** The time taken to complete a single task from submission to completion.

Goal of Parallelism

Database queries are naturally **set-oriented**, making them ideal candidates for parallelization to improve both throughput and response time.

3 Speedup and Scaleup

These represent the two fundamental ways to leverage parallel resources.

3.1 Speedup

Running a **fixed-size task** in less time by increasing the degree of parallelism.

$$\text{Speedup} = \frac{\text{Time on small system}}{\text{Time on larger system}}$$

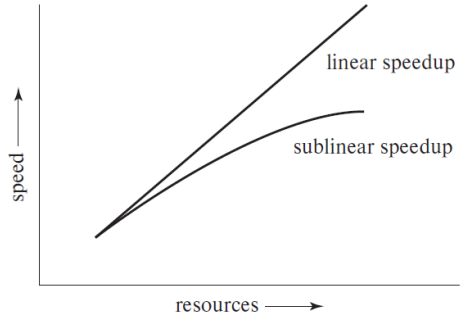

Figure 1: Speedup: Linear vs. Sublinear growth.

3.2 Scaleup

Handling **larger tasks** in the same amount of time by increasing resources proportionally.

$$\text{Scaleup} = \frac{\text{Time for task } Q \text{ on system } M_S}{\text{Time for task } Q_N \text{ on system } M_L}$$

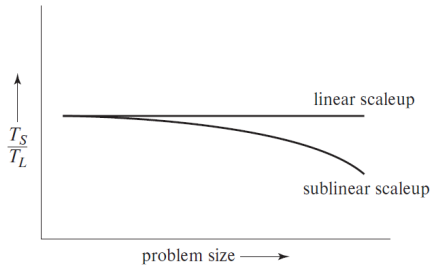

Figure 2: Scaleup: Maintaining constant execution time.

4 Types of Scaleup

- **Batch Scaleup:** Related to decision-support queries. Problem size grows (e.g., scanning a larger relation), and hardware grows to match.
- **Transaction Scaleup:** Related to transaction-processing systems. The transaction rate and database size grow proportionally (e.g., more users/accounts).

5 Factors Limiting Performance

Ideally, parallel systems should exhibit **linear speedup** and **linear scaleup**. In practice, performance is often sublinear due to several limiting factors:

- **Startup Costs:** Initializing and coordinating thousands of processes may take significant time compared to actual computation.
- **Interference:** Concurrent processes compete for shared resources such as buses, disks, caches, and locks.

- **Skew:** Overall execution time is determined by the **slowest task**. Perfectly equal workload distribution is difficult.

Parallel Performance Laws

Amdahl's Law (Speedup Limitation): If a fraction $1-p$ of a program is strictly sequential, the maximum speedup on n processors is:

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{n}}$$

Example: Suppose 90% of a program is parallelizable ($p = 0.9$) and 10% is sequential.

- With $n = 10$ processors:

$$\text{Speedup} = \frac{1}{0.1 + \frac{0.9}{10}} = \frac{1}{0.19} \approx 5.26$$

- Even with **infinite processors**:

$$\text{Speedup}_{\max} = \frac{1}{1-p} = \frac{1}{0.1} = 10$$

Conclusion: The sequential portion fundamentally limits speedup.

Gustafson's Law (Scalable Workloads): When problem size grows with available resources, scaleup on n processors is:

$$\text{Scaleup} = \frac{1}{n(1-p) + p}$$

Example: Assume again that $p = 0.9$ and the problem size increases proportionally with the number of processors.

- With $n = 10$ processors:

$$\text{Scaleup} = \frac{1}{10(0.1) + 0.9} = \frac{1}{1.9} \approx 0.53$$

Interpretation: Although absolute speedup is limited, larger problems can be solved efficiently by increasing resources, making parallelism highly effective for scalable workloads.